CSI101 COMPUTER SYSTEMS

Lab 2 — Logical and Arithmetic Shifts

Assembly Programming · Practical application 11.2 — Logical and arithmetic shift

2-hour lab session · Visual Studio 2022 · Win32 Console Application · x86 platform

Shifting a value right divides it by a power of two — but only if the sign is handled correctly. This lab shows what goes wrong when it is not, then puts shifts to work with a variable count and with the carry flag.

What you will be able to do

- Explain why `SHR` and `SAR` give different answers for a negative value, and choose correctly between them.
- Use a shift count held in `CL` rather than a constant, and load it correctly.
- Read the carry flag after a shift to inspect individual bits.
- Combine `LOOP` with a flag test, knowing that `LOOP` itself changes no flags.

How the session is organised

0:00 – 0:25 Part A. Type the lecture program in, build it, run it, and read the notes.

0:25 – 1:10 Task 1. Extend the program.

1:10 – 1:50 Task 2. Extend it again.

1:50 – 2:00 Work through the checklist at the end and ask about anything that did not build.

> **TYPE IT, DO NOT PASTE IT**
>
> Typing catches the syntax details that reading skips over — the missing comma, the wrong register width, the label with no instruction after it. The solutions at the end of this handout are for checking your work, not for copying.

Before you start

1. Open Visual Studio 2022 and choose **File > New > Project > Console App (C++)**. Give it the name shown at the top of this handout.
2. Set the **Solution Platforms** dropdown on the toolbar to **x86**. It defaults to x64, where the `_asm` keyword is rejected outright with error C4235.
3. Replace the generated `main` with the Part A listing, and make sure `#include <cstdio>` is at the top — `printf` and `scanf_s` come from there.
4. Press **Ctrl + F5** to build and run without the debugger, so the console window stays open when the program finishes.

> **IF THE PLATFORM IS WRONG**
>
> With the platform left at x64 the compiler reports that the `_asm` keyword is not supported on this architecture (C4235). Inline assembly exists on 32-bit x86 and nowhere else.

Part A — the lecture program

Take the value −32, shift it right two places twice — once with SHR and once with SAR — and print both results alongside a left shift. The three results are computed in assembly and printed from C++, which is the division of labour used for the rest of the module.

Bit patterns worth having in front of you

```
start   1111 1111 1111 1111 1111 1111 1110 0000     -32
shr 2   0011 1111 1111 1111 1111 1111 1111 1000      1 073 741 816
sar 2   1111 1111 1111 1111 1111 1111 1111 1000     -8
```

The program

```
#include <cstdio>

void partA()
{
    int value = -32;
    int logical, arithmetic, leftShift;

    printf("--- Part A: SHR, SAR and SHL on %d ---\n\n", value);

    _asm {
        mov     eax, value
        shr     eax, 2              // LOGICAL right shift: zeros come in at the top
        mov     logical, eax

        mov     eax, value
        sar     eax, 2              // ARITHMETIC right shift: the sign bit is copied
        mov     arithmetic, eax

        mov     eax, value
        shl     eax, 1              // left shift by 1 = multiply by 2
        mov     leftShift, eax
    }

    printf("  shr eax, 2  ->  %d\n", logical);       // 1073741816 - sign bit lost
    printf("  sar eax, 2  ->  %d\n", arithmetic);    // -8         - correct
    printf("  shl eax, 1  ->  %d\n", leftShift);     // -64
    printf("\nOnly SAR gives the arithmetically correct -32 / 4 = -8.\n");
    printf("Use SAR for signed values and SHR for unsigned values.\n");
}
```

Expected output

```
  shr eax, 2  ->  1073741816
  sar eax, 2  ->  -8
  shl eax, 1  ->  -64
```

What to notice

- SHR always shifts **zeros** in at the top. For a negative number that destroys the sign bit, and −32 turns into 1,073,741,816.
- SAR copies the existing sign bit into the vacated positions, so −32 becomes −8, which is the arithmetically correct −32 ÷ 4.
- The rule to remember: **'SAR' for signed values, 'SHR' for unsigned values.**
- SHL and SAL are two names for the same instruction. Left shift is the same operation whether the value is signed or not.
- Each shift destroys the value in EAX, so `value` is reloaded before each one.

Task 1 — take the shift count from CL

What to write

- Ask the user for an integer and for a shift count between 0 and 31.
- Produce all three results — `SHR`, `SAR` and `SHL` — using that count instead of a constant.
- Reject a count outside 0 to 31 before you reach the assembly block.

Expected output

```
Enter an integer (try a negative one): -32
Enter a shift count (0 to 31): 3

  value    = -32
  shr by 3  = 536870908
  sar by 3  = -4
  shl by 3  = -256
```

Hints

- A shift count that is not a constant **must** be in `CL`. No other register will do — the instruction encoding has no room for a choice.
- You cannot load a byte register straight from a 32-bit variable. Load the whole value first with `mov ecx, count`; `CL` is then the bottom byte of what you just loaded.
- Reload `value` into `EAX` before each of the three shifts, exactly as Part A does.
- Try a positive value as well. When the sign bit is 0, `SHR` and `SAR` agree — which is precisely why the bug is so easy to miss.

Task 2 — count the 1-bits

What to write

- Ask the user for an integer.
- Count how many of its 32 bits are set to 1, using a loop in assembly.
- Print the count.

Expected output

```
Enter an integer: 255

255 has 8 bit(s) set to 1 out of 32.
(Check: -1 has 32, 0 has 0, 255 has 8, -32 has 27.)
```

Hints

- `shr eax, 1` pushes the bottom bit out of the register and into the **carry flag**. `JNC` then skips the count when that bit was a 0.
- Set the loop up with `mov ecx, 32` — an `int` is 32 bits wide, so a fixed 32 iterations covers every case including 0 and −1.
- `LOOP` changes no flags at all, so it will not disturb the carry test you have just made. `INC` does not touch the carry flag either.
- `EBX` is one of the registers you must preserve. Push it at the top of the block and pop it at the bottom.

Before you leave

- The Solution Platform reads **x86**, not x64.
- Part A reproduces 1073741816, −8 and −64 exactly.
- Task 1 loads `ECX` before using `CL`, and has been tested with both a negative and a positive value.
- Task 2 saves and restores `EBX`.
- The bit count agrees with the four check values: −1 → 32, 0 → 0, 255 → 8, −32 → 27.

If something will not build

- `_asm not supported on this architecture` — the platform is still x64. Change it to x86.
- `improper operand type` — the operand size is ambiguous. Add `DWORD PTR`, or check you have not mixed `eax` with `al`.
- `undeclared identifier` inside a block — a misspelt variable name, or one not in scope. Names are **case sensitive** inside `_asm`, even though instruction mnemonics are not.
- The program builds but crashes *after* the block — the stack is unbalanced, or `EBX`, `ESI`, `EDI` or `EBP` was not restored.
- Right answer, wrong sign — an unsigned jump was used on signed data, or `SHR` where `SAR` was needed.

Solutions

Complete, commented code for both tasks. The Part A listing earlier in this handout is already the full solution for that section. Everything here matches the `CSI101_Lab02_Solution` project in the accompanying Visual Studio package line for line.

Task 1 — take the shift count from CL

```
void task1()
{
    int value = 0;
    int count = 0;
    int logical, arithmetic, leftShift;

    printf("\n--- Task 1: shift by a count held in CL ---\n\n");

    printf("Enter an integer (try a negative one): ");
    if (scanf_s("%d", &value) != 1) return;
    printf("Enter a shift count (0 to 31): ");
    if (scanf_s("%d", &count) != 1) return;

    if (count < 0 || count > 31) {
        printf("A 32-bit shift count outside 0..31 is not meaningful. Stopping.\n");
        return;
    }

    _asm {
        // A shift count that is not a constant MUST be in CL - no other
        // register will do.  CL is the low byte of ECX, and we cannot load a
        // byte register directly from a 32-bit variable, so load ECX first
        // and then use its low byte.
        mov     ecx, count          // ecx = the whole 32-bit count
                                    // cl is now the bottom byte of that value

        mov     eax, value
        shr     eax, cl             // logical  - zeros shifted in
        mov     logical, eax

        mov     eax, value
        sar     eax, cl             // arithmetic - sign preserved
        mov     arithmetic, eax

        mov     eax, value
        shl     eax, cl             // left shift - same instruction either way
        mov     leftShift, eax
    }

    printf("\n  value    = %d\n", value);
    printf("  shr by %-2d = %d\n", count, logical);
    printf("  sar by %-2d = %d\n", count, arithmetic);
    printf("  shl by %-2d = %d\n", count, leftShift);

    if (value < 0)
        printf("\nNegative input: notice SHR and SAR disagree, SAR is the correct divide.\n");
    else
        printf("\nNon-negative input: SHR and SAR agree, because the sign bit is 0.\n");
}
```

Notes on this solution

- The `mov ecx, count` line is doing two jobs: it loads the count, and it puts it somewhere `CL` can reach. Writing `mov cl, count` instead produces an operand size conflict, because `count` is a `DWORD` and `CL` is a `BYTE`.
- On x86 the processor uses only the bottom five bits of `CL` for a 32-bit shift, which is why counts above 31 are meaningless — the range check in C++ saves students a confusing result.

Task 2 — count the 1-bits

```
void task2()
{
    int value = 0;
    int ones  = 0;

    printf("\n--- Task 2: count the 1-bits ---\n\n");
    printf("Enter an integer: ");
    if (scanf_s("%d", &value) != 1) return;

    _asm {
        push    ebx                 // EBX must be preserved - save it first

        mov     eax, value
        xor     ebx, ebx            // ebx = 0, the running count of 1-bits
        mov     ecx, 32             // an int is 32 bits, so 32 iterations

    countLoop:
        shr     eax, 1              // the bit shifted out lands in the CARRY flag
        jnc     noBit               // CF = 0 -> that bit was a 0, skip
        inc     ebx                 // CF = 1 -> count it
                                    // INC does not disturb CF, but we have
                                    //   already used it, so that is fine
    noBit:
        loop    countLoop           // dec ecx, jump back while ecx != 0
                                    // LOOP changes no flags at all

        mov     ones, ebx           // write the count back to C++

        pop     ebx                 // restore on the way out
    }

    printf("\n%d has %d bit(s) set to 1 out of 32.\n", value, ones);
    printf("(Check: -1 has 32, 0 has 0, 255 has 8, -32 has 27.)\n");
}
```

Notes on this solution

- The loop always runs 32 times, even once `EAX` has reached zero. That is deliberate: a version that stops early is faster but needs a second exit condition, and the point here is the carry flag.
- `ADC EBX, 0` — add with carry — would replace the `JNC` and `INC` pair with a single branchless instruction. Try it once the branching version works.